



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Healthcare System using Object-Oriented Design

Object-Oriented Analysis and Design (CSA1104)

Name: Sanna Reddy Poojitha

Reg no: 192320076

Course Code: CSA1104(OOAD)

Faculty name : Dr. A.Gayathri

INTRODUCTION:

Healthcare organizations handle huge amounts of patient information every day.

Traditional paper-based systems are slow, error-prone, and difficult to maintain.

A Smart Healthcare System is a software application that helps hospitals and clinics manage:

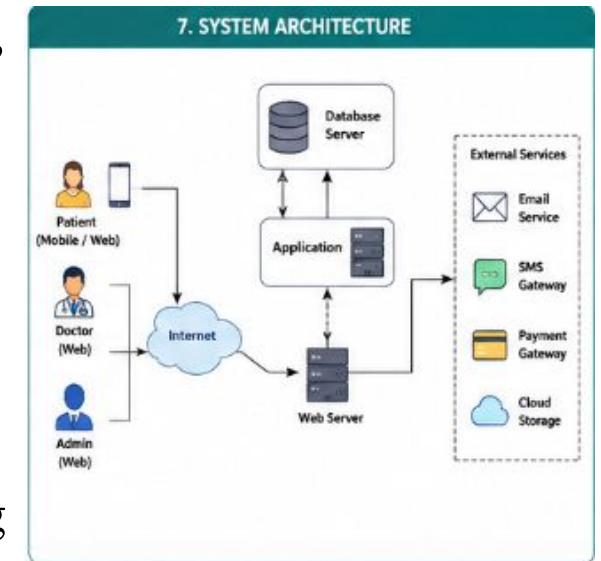
- Patient records
- Doctor information
- Appointments
- Medical history
- Billing
- Prescriptions



Object-Oriented Programming (OOP) provides an efficient way to design such systems.

OBJECTIVE:

- To design a **Smart Healthcare System** using Object-Oriented Programming (OOP) concepts for efficient management of patients, doctors, appointments, and medical records.
- To provide a **secure and centralized platform** for storing, accessing, and updating healthcare information while maintaining patient privacy.
- To improve the **quality and efficiency of healthcare services** by reducing paperwork, minimizing errors, and enabling faster appointment scheduling and record management.
- To develop a **scalable, maintainable, and user-friendly system** that supports future enhancements such as telemedicine, cloud storage, mobile applications, and AI-based healthcare services.



SYSTEM MODULES:

Patient Module

- Patient registration
- Profile management
- Medical history
- View prescriptions

Doctor Module

- Doctor profile
- Specialization details
- Patient consultation
- Diagnosis and treatment

Appointment Module

- Book appointments
- Reschedule appointments
- Cancel appointments

Medical Record Module

- Store diagnosis
- Laboratory reports
- Prescriptions
- Treatment history

Billing Module

- Generate invoices
- Insurance processing
- Online payment

OOP CONCEPTS USED:

Class

Defines the blueprint for system entities such as Patient, Doctor, Appointment, and MedicalRecord.

Object

Represents real-world entities created from classes.

Encapsulation

Protects sensitive patient information by restricting direct access and allowing controlled access through methods.

Inheritance

Allows common properties to be inherited from the Person class by Patient and Doctor classes.

Polymorphism

Allows the same function to perform different operations depending on the object.

Abstraction

Displays only essential functions to users while hiding the internal implementation details.

UML Use Case Diagram:

Explanation

The Use Case Diagram illustrates how different users interact with the Smart Healthcare System.

Patient

- Register/Login
- Book Appointment
- View Medical Records
- Pay Bills

Doctor

- View Patient Details
- Update Medical Records
- Write Prescription

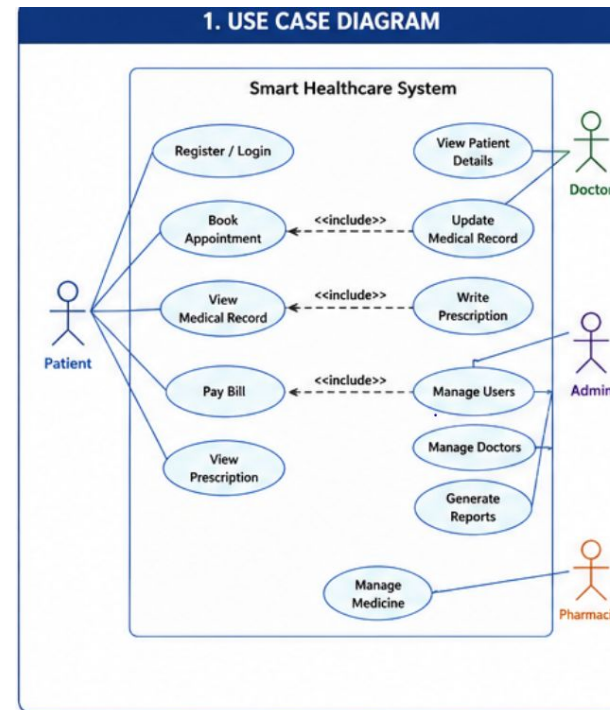
Administrator

- Manage Users
- Manage Doctors
- Generate Reports

Pharmacist

- Manage Medicines

This diagram helps identify system functionalities and user interactions



UML Class Diagram:

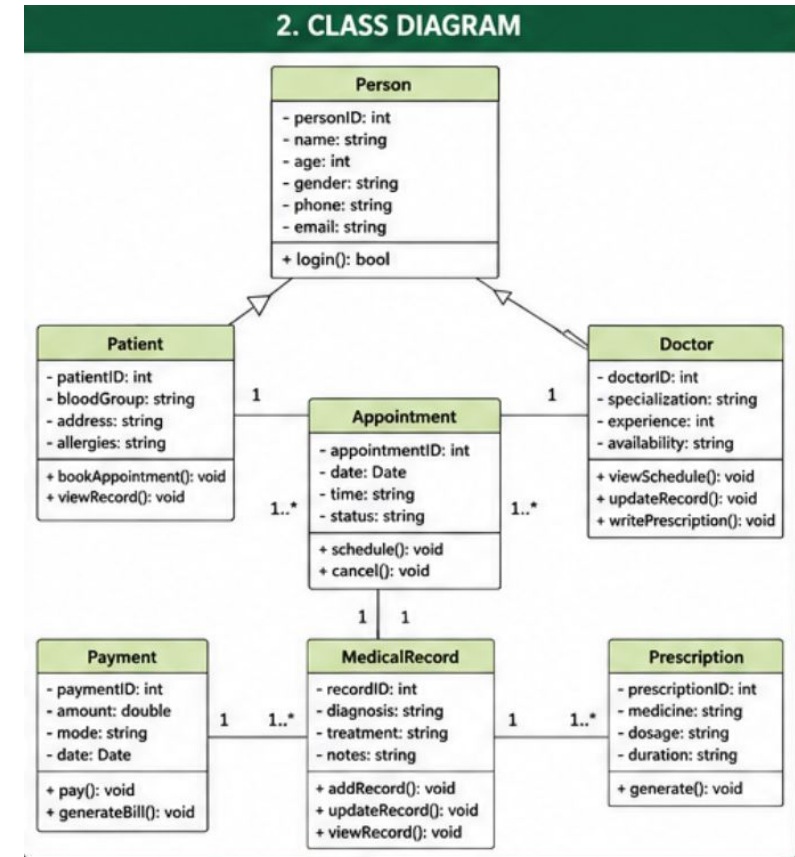
The Class Diagram represents the structure of the system.

Main classes include:

- Person
- Patient
- Doctor
- Appointment
- MedicalRecord
- Payment
- Prescription

Relationships show how objects communicate and share information.

Inheritance reduces code duplication, while associations define interactions among classes.



UML Sequence Diagram:

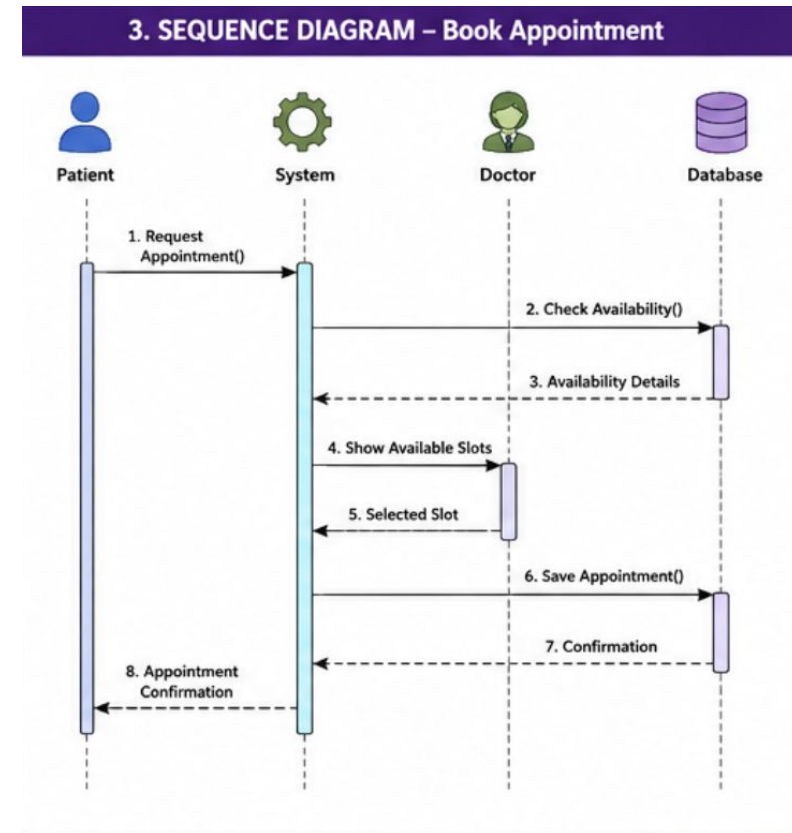
Explanation

The Sequence Diagram illustrates the process of booking an appointment.

Steps:

1. Patient sends appointment request.
2. System checks doctor availability.
3. Doctor confirms availability.
4. Appointment is stored in the database.
5. Confirmation is sent to the patient.

This diagram shows the order of communication between different system components.



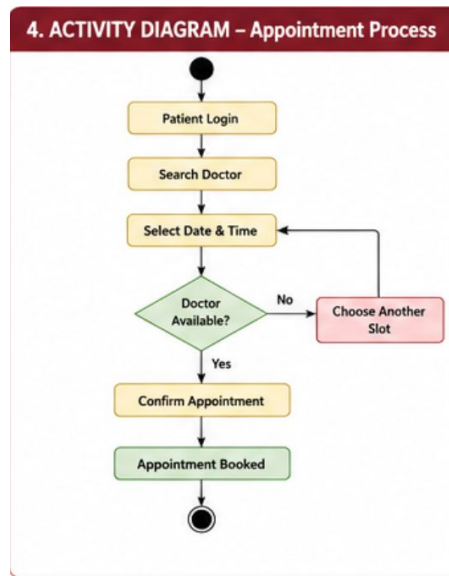
UML Activity Diagram

The Activity Diagram explains the workflow followed during appointment booking.

Process:

- Patient Login
- Search Doctor
- Select Date & Time
- Check Doctor Availability
- Confirm Appointment
- Update Medical Records
- Generate Bill

Decision nodes are used to verify doctor availability before confirming the appointment.

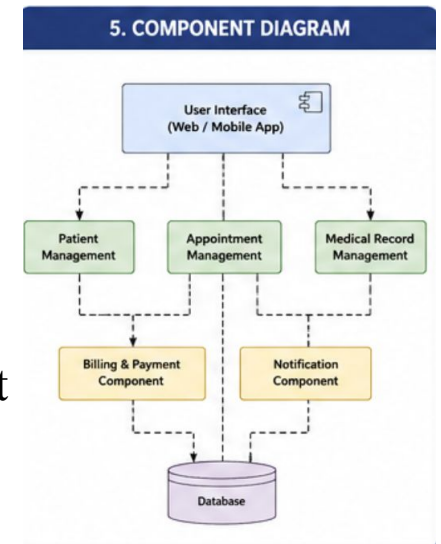


UML Component Diagram

The Component Diagram shows the software modules of the healthcare system.

Components include:

- User Interface
- Patient Management
- Appointment Management
- Medical Record Management
- Billing System
- Notification Service
- Database



These components communicate with each other to perform healthcare operations efficiently

UML Deployment Diagram

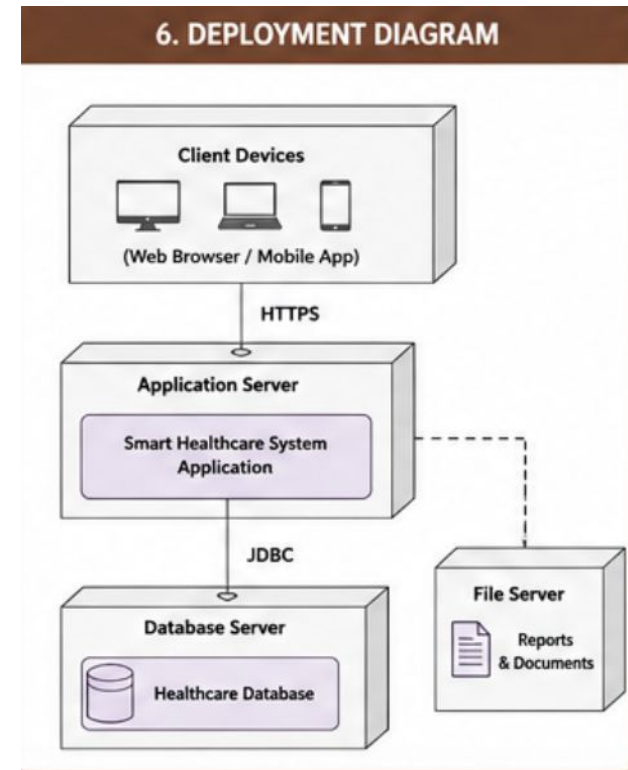
Explanation

The Deployment Diagram illustrates how the application is deployed on hardware devices.

Hardware Nodes:

- Client Devices
- Application Server
- Database Server
- File Server

Communication occurs through secure internet protocols such as HTTPS.



SDLC Phases

Software Development Life Cycle

Requirement Analysis

Collect requirements from hospitals and users.

System Design

Design UML diagrams, database, and software architecture.

Development

Develop the application using OOP concepts.

Testing

Perform unit testing, integration testing, and system testing.

Deployment

Install the software in hospitals and healthcare centers.

Maintenance

Provide updates, bug fixes, and security improvements.

Advantages

- Digital management of patient records
- Faster appointment scheduling
- Improved healthcare services
- Reduced paperwork
- Enhanced data security
- Easy report generation
- Better communication between doctors and patients
- Scalable and maintainable software
- Faster access to medical history
- Improved decision-making

CONCLUSION

- The **Smart Healthcare System** provides an efficient and digital solution for managing healthcare services such as patient records, appointments, prescriptions, and billing.
- Object-Oriented Programming (OOP)** concepts like **Encapsulation, Inheritance, Polymorphism, and Abstraction** make the system secure, reusable, scalable, and easy to maintain.
- UML diagrams** help visualize the system structure, user interactions, workflows, and deployment, making software design easier to understand and develop.
- The **Software Development Life Cycle (SDLC)** ensures systematic development through planning, design, implementation, testing, deployment, and maintenance.
- The system improves **hospital efficiency**, reduces paperwork, minimizes human errors, and enhances the quality of patient care.
- It supports future technologies such as **Artificial Intelligence (AI), Cloud Computing, Telemedicine, and IoT**, making it suitable for next-generation healthcare applications.

THANK YOU